Exercício 9.2: Usuário por certificado com kubeconfig funcional

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1. A CA do kind assina CSRs `kubernetes.io/kube-apiserver-client`, o que o EKS não faz.
- Sequência completa em `01-commands.sh`.
- `joao-qa.key` e `kubeconfig-joao.yaml` contêm a chave privada e **não** vão para o repositório, que é público (estão no `.gitignore`). Ficam só na máquina local.

Emissão da identidade

```
$ openssl genrsa -out joao-qa.key 2048
$ openssl req -new -key joao-qa.key -out joao-qa.csr -subj "/CN=joao.qa/O=fiap:qa"
$ openssl req -in joao-qa.csr -noout -subject
subject=CN=joao.qa, O=fiap:qa
```

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: joao.qa
spec:
  request:
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSBSRVFVRVNULS0tLS0KTUlJQ2FUQ0NBVkVDQVFBd0pERVFNQTRHQTFVRUF3d0hhbTloYnk1e
FlURVFNQTRHQTFVRUNnd0habWxoY0RweApZVENDQVNJd0RRWUpLb1pJaHZjTkFRRUJCUUFEZ2dFUEFEQ0NBUW9DZ2dFQkFLNWkxRk
d6M2tsU014TllrWm1tCmV2MHhNbjFkaGhvSTljUk1MTWY4RU5ySVovOWlIZElmdzNTL1U5Wit3SXZ4WFdZNDNwRWlIK2RoUDJyVGd
BTE4KRlh6bzlROTNBQnpScW4vaEZGYjlCMEpyVHVMTC8xemE2cTBKWWpIREVndGxhT2hkQnMzNUk3N0xtTFloVkdNTAphcWFFNTYx
VWhFTzFLMnZKSmFTeHFaUHdWaTI3VXYwM3JCL0tiSXhjWTltZjR6bTFwUWJoZHVIRTkzRC9tOHlZCmJrUGM0WEozVTB2K0RlN080e
GJBTjJxVzJqOTV3ZWx4RFMyOGFuYVBXdXlobXYvbC90RDNqd0ZBV3ZKbjB0QzAKbVFYTmlLYVdkaGhOS0Z2WE1PcVNPa1lBc3l1Rj
VCblU3Ump3QmRPUWpsdmVSL0tXa2ZEdGZNL2RkTmIxdkQvUgpJa1VDQXdFQUFhQUFNQTBHQ1NxR1NJYjNEUUVCQ3dVQUE0SUJBUUJ
hUWovbzk0amtZVHBtbXpqR1hLK3l1Qmw5CnpBOWh6b3dZNnhsV1E0Q25kSVA3SkRic3VHOU9DZDdsQVhiY0xZWERWNzVMNzI5cUov
Q1N1K0JHQTArV2lXSmMKRmlyaHcxSW9DdEhGcDdRVk5MV0tsSUFiQytSUWFaWnFlN0FtUldEVk92Rkt2R1MrV21iT3dJU3NuajlNR
U94RwpWeG1Tb2IwcWRzTUFVWWQ0SEh1OFFwTXNidHExdk5idEJyUWtFUHdnb2c1azVJRXIzcVU4WVZTVFhibnNqUVR6CjhzQllSMm
ZncHRMUFF0VURQYjJGTVFOK3JMcW53V3JlZVY4SUt1T3R0WTZEcG5pdXRLMkNlSGRtek1FK0JLZW0KNTdLUXp0NUE3eUozeHFlMFF
IQ1dpTWo3RzNtQUVNc0RaQmwwQXM3alZoNnFueHRJK3p1SiszeisrSVRaCi0tLS0tRU5EIENFUlRJRklDQVRFIFJFUVVFU1QtLS0t
LQo=
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 604800 # 7 dias
  usages:
    - client auth
```

```
$ kubectl apply -f csr-joao.yaml
certificatesigningrequest.certificates.k8s.io/joao.qa created
$ kubectl get csr joao.qa
NAME      AGE   SIGNERNAME                            REQUESTOR          REQUESTEDDURATION
CONDITION
joao.qa   0s    kubernetes.io/kube-apiserver-client   kubernetes-admin   7d                  Pending
$ kubectl certificate approve joao.qa
certificatesigningrequest.certificates.k8s.io/joao.qa approved
$ kubectl get csr joao.qa
NAME      AGE   SIGNERNAME                            REQUESTOR          REQUESTEDDURATION
CONDITION
joao.qa   0s    kubernetes.io/kube-apiserver-client   kubernetes-admin   7d
Approved,Issued

$ kubectl get csr joao.qa -o jsonpath='{.status.certificate}' | base64 -d > joao-qa.crt
$ openssl x509 -in joao-qa.crt -noout -subject -dates
subject=O=fiap:qa, CN=joao.qa
notBefore=Oct 24 03:44:08 2026 GMT
notAfter=Oct 31 03:44:08 2026 GMT
$ openssl x509 -in joao-qa.crt -noout -subject -nameopt RFC2253
subject=CN=joao.qa,O=fiap:qa
```

Duas observações:

- O signer do Kubernetes reordenou o subject: o CSR saiu com `CN, O` e o certificado voltou com `O, CN`. Para o API server a ordem não importa, porque ele lê `CN` como usuário e cada `O` como grupo.
- `notBefore` está 5 min antes da emissão (00:49 local = 03:49 UTC). O signer recua o início da validade para tolerar diferença de relógio entre máquinas. Duração: exatamente 7 dias, igual ao `expirationSeconds: 604800`.

Kubeconfig autocontido: os três blocos estão embutidos (`certificate-authority-data`, `client-certificate-data`, `client-key-data`), sem nenhum caminho de arquivo. Teste copiando só ele para um diretório vazio:

```
$ mkdir teste && cp kubeconfig-joao.yaml teste/ && cd teste
$ kubectl --kubeconfig=./kubeconfig-joao.yaml auth whoami
ATTRIBUTE                                           VALUE
Username                                            joao.qa
Groups                                              [fiap:qa system:authenticated]
Extra: authentication.kubernetes.io/credential-id   [X509SHA256=2b758edd74ea...]
```

(a) Antes de qualquer binding: 403, não 401

```
$ kubectl --kubeconfig=./kubeconfig-joao.yaml get pods -n store-dev
Error from server (Forbidden): pods is forbidden: User "joao.qa" cannot list resource "pods" in API
group "" in the namespace "store-dev"

$ kubectl --kubeconfig=./kubeconfig-joao.yaml get pods -n store-dev -v=6
"Response" verb="GET" url="https://127.0.0.1:34485/api?timeout=32s" status="200 OK"
"Response" verb="GET" url="https://127.0.0.1:34485/apis?timeout=32s" status="200 OK"
"Response" verb="GET" url="https://127.0.0.1:34485/api/v1/namespaces/store-dev/pods?limit=500"
status="403 Forbidden"
```

Para contrastar, um certificado **autoassinado** com o mesmo subject, fora da CA do cluster:

```
$ kubectl --kubeconfig=./kubeconfig-falso.yaml get pods -n store-dev
error: You must be logged in to the server (Unauthorized)
"Response" verb="GET" url="https://127.0.0.1:34485/api?timeout=32s" status="401 Unauthorized"
```

Diferença: o API server faz duas etapas em sequência. **Autenticação** responde "quem é você?": o certificado assinado pela CA do cluster passa, e a identidade vira `joao.qa` com grupo `fiap:qa`. Quando essa etapa falha, a resposta é **401 Unauthorized** e a requisição para ali, como no certificado falso. **Autorização** (RBAC) responde "o que você pode fazer?": se nenhum binding permite, a resposta é **403 Forbidden**. Receber

403 prova que a identidade foi aceita e só falta permissão. Note que o João já recebe `200` em `/api` e `/apis` (discovery, liberado para `system:authenticated`), mas `403` em `pods`.

(b) RoleBinding para o grupo, não para o usuário

```yaml
# (b) leitura para o GRUPO fiap:qa (vem do O= do certificado), so em store-dev
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: qa-view
  namespace: store-dev
subjects:
  - kind: Group
    name: fiap:qa
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view
---
# (c) exec apenas em store-dev
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: qa-exec
  namespace: store-dev
rules:
  - apiGroups: [""]
    resources: ["pods/exec"]
    verbs: ["create"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: qa-exec
  namespace: store-dev
subjects:
  - kind: Group
    name: fiap:qa
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: qa-exec
```

```
$ kubectl apply -f rbac-qa.yaml
rolebinding.rbac.authorization.k8s.io/qa-view created
role.rbac.authorization.k8s.io/qa-exec created
rolebinding.rbac.authorization.k8s.io/qa-exec created

$ kubectl --kubeconfig=./kubeconfig-joao.yaml get pods -n store-dev
NAME                                READY   STATUS    RESTARTS   AGE
sincroniza-catalogo-29880225-kjvd7   1/1     Running   0          4m42s

$ kubectl auth can-i list pods -n store-dev --as=joao.qa
no
$ kubectl auth can-i list pods -n store-dev --as=joao.qa --as-group=fiap:qa
yes
```

Com o mesmo usuário, o resultado muda só pelo grupo: sem `fiap:qa` é `no` e com ele é `yes`. A permissão vem do `O=fiap:qa` do certificado, e nenhum binding cita `joao.qa`. Um segundo QA com outro certificado `O=fiap:qa` herda o mesmo acesso sem mudar nada no RBAC.

(c) `pods/exec` só em `store-dev`

```
$ kubectl --kubeconfig=./kubeconfig-joao.yaml exec -n store-dev sincroniza-catalogo-29880225-kjvd7 --
sh -c 'echo exec ok em $(hostname)'
exec ok em sincroniza-catalogo-29880225-kjvd7

$ kubectl --kubeconfig=./kubeconfig-joao.yaml exec -n store-prod alvo -- sh -c 'echo exec ok'
Error from server (Forbidden): pods "alvo" is forbidden: User "joao.qa" cannot get resource "pods" in
API group "" in the namespace "store-prod"

$ kubectl --kubeconfig=./kubeconfig-joao.yaml auth can-i create pods --subresource=exec -n store-dev
yes
$ kubectl --kubeconfig=./kubeconfig-joao.yaml auth can-i create pods --subresource=exec -n store-prod
no
```

O `kubectl exec` faz primeiro um `GET` do Pod e só depois o `POST` em `pods/exec`. Em `store-prod` ele já para no `get pods`, por isso o erro cita `get` e não `exec`. O `can-i` com `--subresource=exec` confirma que o `exec` em si também é negado. Um detalhe: `kubectl auth can-i create pods/exec` (com barra) respondeu `no` mesmo em `store-dev`. Para subresource, a forma correta é `--subresource=exec`.

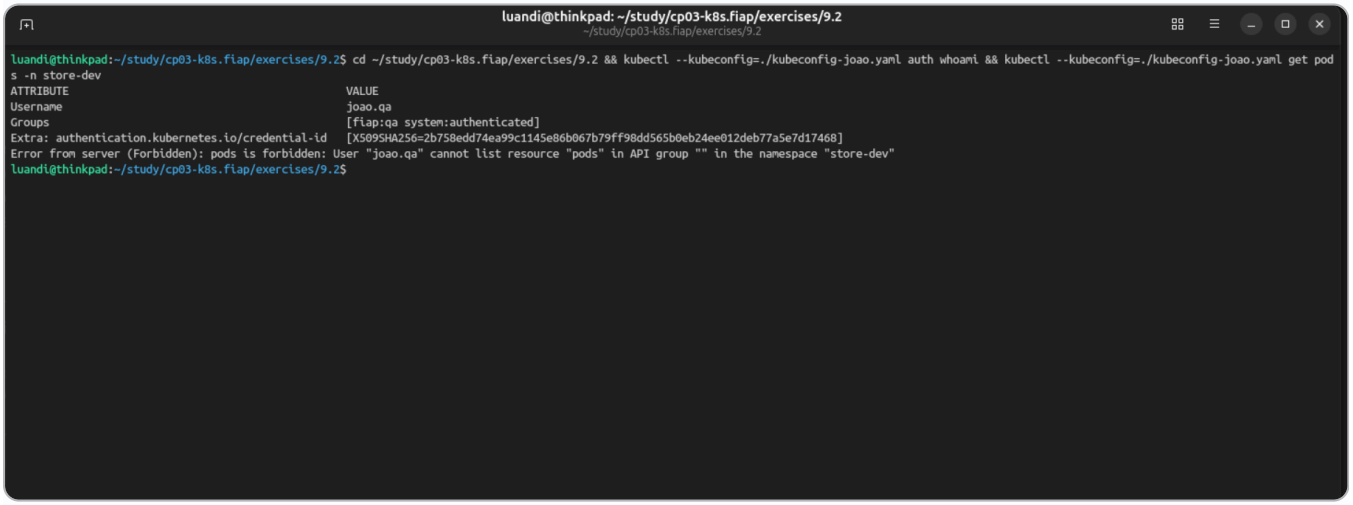
(d) Revogação: continua 403

```
$ kubectl delete -f rbac-qa.yaml
rolebinding.rbac.authorization.k8s.io "qa-view" deleted from store-dev namespace
role.rbac.authorization.k8s.io "qa-exec" deleted from store-dev namespace
rolebinding.rbac.authorization.k8s.io "qa-exec" deleted from store-dev namespace

$ kubectl --kubeconfig=./kubeconfig-joao.yaml get pods -n store-dev
Error from server (Forbidden): pods is forbidden: User "joao.qa" cannot list resource "pods" in API
group "" in the namespace "store-dev"
url="https://127.0.0.1:34485/api/v1/namespaces/store-dev/pods?limit=500" status="403 Forbidden"

$ kubectl --kubeconfig=./kubeconfig-joao.yaml auth whoami
Username                                        joao.qa
Groups                                          [fiap:qa system:authenticated]
```

Apagar os bindings tira a **autorização**, mas o certificado continua válido até `Oct 31 03:44:08 2026 GMT` e segue **autenticando**. O Kubernetes não tem revogação de certificado de cliente (não há CRL nem OCSP no API server). As saídas reais são: certificado de vida curta (por isso os 7 dias), remover os bindings do grupo, ou, no limite, trocar a CA do cluster.



Critérios de aceite

Critério	Resultado
Kubeconfig em diretório vazio lista Pods de `store-dev`	ok
Subject `CN = joao.qa, O = fiap:qa`	ok (RFC2253: `CN=joao.qa,O=fiap:qa`)

Critério	Resultado
`exec` funciona em `store-dev` e retorna `Forbidden` em `store-prod`	ok
Após revogação, `Forbidden` (403) e não `Unauthorized` (401)	ok